GitHub Global —— 技术方案设计文档（TechDesign）

施工蓝图。本文件与 `docs/PRD.md` v1.1 成对使用：PRD 回答“做什么”，本文回答“怎么做”。

文档信息

项目	内容
文档类型	技术方案设计（Technical Design）
文档版本	v1.2（M0 实施对齐：Next.js 16 / Prisma 6 / Tailwind v4；正文见 §11）
对齐 PRD	`docs/PRD.md` v1.1
技术栈锁定	`.cursor/rules/project-context.mdc`
代码规范	`.cursor/rules/code-style.mdc`
负责人	全栈架构师 / 开发者本人
最后更新	2026-04-23

---

0. 设计原则（先于一切具体设计）

1. 单体优先（**Modular Monolith**）：MVP 不做微服务，但在 `lib/` 内按领域（github / translator / queue / auth）强边界拆分，后期任一模块可平滑抽出为独立服务。
2. 边界即合约：所有跨模块调用必须走 `lib/*/index.ts` 导出的显式函数签名，不允许跨模块 import 内部文件。
3. 平台抽象：`lib/github/` 之上再抽一层 `lib/vcs/`（P1 引入），GitHub 仅是 VCS 的一种实现，为 F-25（GitLab / Gitee 适配）打底。
4. 数据库即队列（**MVP** 阶段）：用 `TranslationJob.status` + 轮询 worker 模拟队列，不引入 Redis/Upstash，降低基础设施成本（对齐 PRD 6.3）。
5. 幂等优先：所有外部副作用操作（创建分支、创建 PR、Webhook 处理）必须可重试、可去重，用 `externalId` + 唯一约束兜底。
6. 秘钥零硬编码：`process.env.*` 是唯一读取入口，封装在 `lib/env.ts` 里用 zod 校验（启动即失败优于运行时崩）。

---

1. 系统架构

1.1 总体架构图（C4 Context + Container 合并版）

```
flowchart TB
    subgraph User["👤 用户 (开源作者 / DevRel)"]
        Browser["浏览器"]
    end
```

```
    subgraph GH_Platform["🐙 GitHub 平台"]
        GH_OAuth["OAuth Server"]
        GH_App["GitHub App<br/>(Installations)"]
        GH_API["REST / GraphQL API"]
        GH_Webhook["Webhook Publisher"]
    end

    subgraph Vercel["☁ Vercel (部署单元)"]
        subgraph NextApp["Next.js 16 App"]
            direction TB
            UI["App Router 页面<br/>Server Components"]
            ApiRoutes["API Routes<br/>/api/*"]
            Actions["Server Actions"]
            Middleware["middleware.ts<br/>(会话与 Webhook 签名)"]
        end
        Cron["Vercel Cron<br/>(每分钟拨一下 worker)"]
    end

    subgraph Neon["🗄 Neon Postgres"]
        DB[("Prisma Schema<br/>User / Installation<br/>Repository / Job /
FileTranslation")]
    end

    subgraph OR_Platform["🤖 OpenRouter"]
        LLM["多模型路由<br/>(:free / 付费)"]
    end

    Browser -->|"HTTPS"| UI
    UI -->|"Server Action"| Actions
    UI -->|"fetch / SSE"| ApiRoutes
    Actions -->|"Prisma"| DB
    ApiRoutes -->|"Prisma"| DB
    ApiRoutes -->|"Octokit"| GH_API

    Browser -->|"OAuth 登录跳转"| GH_OAuth
    GH_OAuth -->|"code 回调"| ApiRoutes
    Browser -->|"引导安装"| GH_App
    GH_Webhook -->|"push / installation 事件"| Middleware
    Middleware --> ApiRoutes

    ApiRoutes -->|"OpenAI 兼容协议"| LLM
    Cron -->|"HTTP 调用 /api/worker/tick"| ApiRoutes
    ApiRoutes -->|"入队/出队/更新状态"| DB

    classDef ext fill:#fef3c7,stroke:#f59e0b
    classDef core fill:#dbeafe,stroke:#2563eb
    classDef data fill:#dcfce7,stroke:#16a34a
```

```
class GH_Platform,OR_Platform ext
class Vercel,NextApp,Cron core
class Neon,DB data
```

1.2 请求维度：三条关键链路的时序图

本节三张时序图描述的是：请求谁先找谁、按什么顺序、为了什么目的。竖线（参与者）是「角色」；横线箭头是「一次交互」；`alt` / `loop` 表示分支与重复。

1.2.0 新手阅读指南：三条链路分别解决什么？

编号	链路	一句话	你作为用户能感知到什么	与后面 §1.3 的关系
1.2.1	登录 + 授权仓库	证明「你是哪个 GitHub 用户」，并让本应用安全地获得操作你仓库的许可（OAuth + GitHub App 安装）。	点登录 → 跳 GitHub → 回到本站 → 可能看到「去安装 App」→ 装完后看到仓库列表。	主要经过 `app/` 的页面与 `/api/auth/*`、`lib/auth`、`lib/github`；结果写入 DB 的 User / Installation / Repository。
1.2.2	翻译任务与 PR	你点「开始翻译」后，后台慢慢干活（拉文件、调模型、推分支、开 PR），前端只轮询进度。	选仓库与语言 → 出现进度 → 完成后跳到 PR 链接。	`app/` 发起任务；`lib/translator`、`lib/queue`（worker）、`lib/github`、OpenRouter；DB 记 Job / 文件级状态（设计见 §3.2 全量 schema；M0 可能尚未建全表）。
1.2.3	增量翻译（Webhook）	仓库里有人 **push** 了新 **commit** 时，GitHub 主动通知我们；我们先快速收条，再让 Worker 慢慢比对、决定是否新开翻译任务。	你改文档并 push 后，无需再点一次翻译，系统可自动跟进（若已开启）。	`middleware` 验签；`/api/webhooks/github` 入库 WebhookEvent；Worker 消费；依赖 `lib/github` 比对变更。

时序图里常见符号（对照看图）

符号	含义
`->>` / `-->>`	实线：请求；虚线箭头：响应（异步返回常用 `-->>`）。
`actor` / `participant`	人/浏览器/服务/数据库等「竖条」角色。
`autonumber`	给箭头自动编号，方便下文逐步对照。
`alt` / `else` / `end`	条件分支：满足 A 走一支，否则走另一支。
`loop` / `end`	重复：例如「每 2 秒问一次进度」。
`Note over X`	注释：说明背景（不单独占一步请求）。

GitHub 渲染注意：本地预览正常但 github.com 上整块仍是代码时，多为 Mermaid 语法踩坑。必避项包括：时序图消息里不要出现英文分号 **;**；不要用尖括号占位（如 `<sha>`）；不要用 **id[/多段路径/]** 梯形节点（路径里的 **/** 会破坏解析）；`flowchart` 标签里少用 **&**；`erDiagram` 避免 **FK,UK** 这类逗号拼接属性行。完整清单见 **.cursor/rules/docs-mermaid-github.mdc**。

1.2.1 登录 + 授权仓库（首次上手）

```
sequenceDiagram
    autonumber
    actor U as 用户
    participant B as 浏览器
    participant N as Next.js
    participant GH as GitHub
    participant DB as Neon

    U->>B: 点击"用 GitHub 登录"
    B->>N: GET /api/auth/github/start
    N-->>B: 302 → github.com/oauth/authorize
    B->>GH: 授权页
    U->>GH: 同意授权
    GH-->>B: 302 → /api/auth/github/callback?code=xxx
    B->>N: GET /api/auth/github/callback?code=xxx
    N->>GH: POST /login/oauth/access_token
    GH-->>N: access_token
    N->>GH: GET /user
    GH-->>N: user profile
    N->>DB: upsert User
    N->>B: Set-Cookie session=jwt, 302 → /dashboard
    B->>N: GET /dashboard
    N->>DB: 查 Installations 是否存在
    alt 无 Installation
        N-->>B: 渲染"去安装 GitHub App"引导
        U->>GH: 安装 App 到选定仓库
```

```
        GH->>N: POST /api/webhooks/github (installation.created)
        N->>DB: upsert Installation + Repository[]
        GH-->>B: 302 → /dashboard
    else 已有 Installation
        N-->>B: 渲染仓库列表
    end
```

1.2.1 在做什么（白话）

你要用本应用操作 GitHub 上的文档，必须先完成两件事：（**A**）登录——让本站知道你是 GitHub 上的谁；（**B**）授权——让本站能以 **GitHub App** 的身份访问你选中的仓库。OAuth 负责（A），安装 App 到仓库负责（B）。数据库里会记下你的 `User`、App 的 `Installation` 以及可见的 `Repository` 列表。

参与者分工

角色	是什么	在本链路中的作用
用户	真人	点登录、在 GitHub 上点同意、安装 App。
浏览器	Chrome 等	替你发 HTTP 请求、保存 Cookie、跟随 302 跳转。
Next.js	本站的 App + API	发起 OAuth、换 token、建会话、读库、渲染页面。
GitHub	github.com	发 OAuth 页面、发 token、在用户安装 App 后推送 Webhook。
Neon	Postgres 数据库	存 User / Installation / Repository，下次请求可复用。

建议对照上图的步骤（按阶段理解，不必死记编号）

阶段	发生了什么	作用
OAuth 启动	用户点登录 → 浏览器访问 `/api/auth/github/start` → **302** 到 GitHub 授权页	把用户交给 GitHub 做身份与授权确认，本站不碰密码。
用户同意	用户在 GitHub 点授权 → GitHub **302** 回 `callback?code=`	`code` 是一次性「兑换券」，只能服务端用来换 `access_token`。
换 token + 拉资料	Next.js 用 `code` 换 `access_token`，再 `GET /user`	拿到稳定的用户标识（如 `githubId`、login），准备写入 `User` 表。
落库 + 会话	`upsert User`，`Set-Cookie` 后 **302** 到 `/dashboard`	以后浏览器带 Cookie，本站就知道「你是谁」。
进控制台	`GET /dashboard`，查是否已有 **Installation**	仅有 OAuth 不能代表 App 已装到仓库；所以要分支判断。
分支：未安装 App	页面引导去 GitHub 安装；用户装完后 GitHub 发 `installation` 类 Webhook；本站 `upsert Installation + Repository[]`	把「哪些仓库可被本 App 访问」同步进数据库。
分支：已安装	直接渲染仓库列表	用户可继续选仓库做翻译等操作。

与 1.2.2 / 1.2.3 的关系

没有 1.2.1，后续既无法代表用户调 GitHub API，也收不到带安装上下文的 Webhook，翻译与增量链路都不会成立。

```
flowchart TB
    subgraph login["阶段 A: OAuth 登录"]
        L1[用户在站点点登录]
        L2[跳转 GitHub 授权]
        L3[带回 code 回调本站]
        L4[换 token 并写入 User + Cookie]
        L1 --> L2 --> L3 --> L4
    end
    subgraph install["阶段 B: GitHub App 安装（首次）"]
        I1[打开控制台 dashboard]
        I2{库里有 Installation 吗}
        I3[引导去 GitHub 安装 App]
        I4[Webhook 同步仓库列表到 DB]
        I5[展示已授权仓库列表]
```

```
        I1 --> I2
        I2 -->|否| I3 --> I4 --> I5
        I2 -->|是| I5
    end
    login --> install
```

1.2.2 翻译任务 & PR 产物

```
sequenceDiagram
    autonumber
    actor U as 用户
    participant B as 浏览器
    participant N as Next.js
    participant W as Worker<br/>(/api/worker/tick)
    participant GH as GitHub
    participant OR as OpenRouter
    participant DB as Neon

    U->>B: 选仓库 + 勾语言, 点"开始翻译"
    B->>N: Server Action: createJob
    N->>DB: INSERT TranslationJob (status=pending)<br/>+
FileTranslation(status=queued) × N
    N-->>B: 返回 jobId
    loop 每 2s 轮询直到终态
        B->>N: GET /api/jobs/{id}
        N-->>B: { progress, files[] }
    end

    Note over W: Vercel Cron 每 60s 拨一次
    W->>DB: SELECT pending jobs FOR UPDATE SKIP LOCKED
    W->>DB: UPDATE status=running
    W->>GH: 列出 .md 文件 (Git Tree API)
    loop 每个文件 × 每种语言
        W->>GH: 取文件原文
        W->>OR: POST /chat/completions
        OR-->>W: 译文
        W->>W: AST 校验 (remark) ❗
        W->>GH: 创建/追加 commit 到 i18n/lang-时间戳 分支
        W->>DB: UPDATE FileTranslation: status=succeeded, sourceHash, tokens
    end
    W->>GH: 创建/更新 PR
    W->>DB: UPDATE TranslationJob status=succeeded, prUrl
    B-->>U: 下一次轮询拿到 succeeded, 跳转 PR
```

1.2.2 在做什么（白话）

用户一次「开始翻译」会立刻在数据库里创建一条（或多条）任务记录，但真正耗时的活（读 GitHub 文件、调 LLM、写回分支、开 PR）在 **Worker** 里异步做。浏览器用短轮询（例如每 2 秒）问「好了没」，避免在 serverless 上长时间挂一个连接。Worker 由 **Vercel Cron** 周期性触发（例如每分钟），从 DB 里抢待处理任务，防止多实例重复执行同一任务。

参与者分工

角色	是什么	在本链路中的作用
用户 / 浏览器	操作者与前端	提交「选仓库 + 语言」；轮询 `/api/jobs/:id` 拿进度。
Next.js	Server Action + API	`createJob` 写 DB；提供任务查询接口。
Worker	本设计里多为 `/api/worker/tick`	被 Cron 调用；拉 GitHub 树、调 OpenRouter、推 commit、更新 PR、回写 DB。
GitHub	远程仓库与 API	提供文件内容与写分支 / PR 的能力。
OpenRouter	LLM 网关	按 OpenAI 兼容协议返回译文。
Neon	数据库	记录 Job、每个文件的翻译状态、token 用量等。

两条「时间线」并行（理解的关键）

时间线	谁发起	在干什么
用户线（前台）	浏览器	Server Action 创建任务 → **loop** 轮询进度 → 终态后跳转 PR。
Worker 线（后台）	Cron → Worker	从 DB 取 `pending` → `running` → 对每个文件 × 语言调模型 → 校验 Markdown AST → 推到 `i18n/...` 分支 → 开/更新 PR → Job `succeeded`。

Worker 内层循环在干什么（逐步）

步骤	动作	目的
1	`SELECT ... FOR UPDATE SKIP LOCKED`	多实例时同一任务只被一个 **Worker** 拿走。
2	Git Tree API 列 `.md`	知道要翻译哪些文件。
3	每个文件拉原文 → OpenRouter → 得译文	真正翻译。
4	remark AST 校验	防止模型破坏 Markdown 结构（表格/代码块等）。
5	commit 到专用分支	不直接污染默认分支，用 PR 给人审。
6	更新 `FileTranslation` / `TranslationJob`	进度可查询、可计费、可重试。

与 1.2.1 / 1.2.3 的关系
依赖 1.2.1 拿到的权限才能读写仓库；1.2.3 则在源文件变更时可能自动再触发类似本链路的任务（通常新建 Job，而不是让用户再点一次）。

```
flowchart LR
    subgraph front["前台: 用户浏览器"]
        F1[Server Action 创建 Job]
        F2[轮询 GET /api/jobs/id]
    end
    subgraph back["后台: Cron + Worker"]
        B1[tick 取 pending Job]
        B2[读 GitHub 文件]
        B3[OpenRouter 翻译]
        B4[写分支 + PR]
        B1 --> B2 --> B3 --> B4
    end
    subgraph db["DB"]
        D[(Job / File 状态)]
    end
    F1 --> D
    F2 --> D
    B1 --> D
    B4 --> D
```

1.2.3 增量翻译（Webhook 驱动）

```
sequenceDiagram
    autonumber
    participant GH as GitHub
    participant MW as middleware.ts
    participant N as /api/webhooks/github
```

```
participant DB as Neon
participant W as Worker

GH->>MW: POST /api/webhooks/github<br/>X-Hub-Signature-256
MW->>MW: 校验 HMAC-SHA256
alt 签名非法
    MW-->>GH: 401
end
MW->>N: 转发
N->>N: 3s 内早返回（防 GitHub 10s 超时）
N->>DB: INSERT WebhookEvent (raw payload, status=pending)
N-->>GH: 200 OK
Note over W: Worker 异步消费 WebhookEvent
W->>DB: SELECT pending WebhookEvent
alt event = push
    W->>GH: Compare API: 取变更文件列表
    W->>DB: 与 FileTranslation.sourceHash 比对
    W->>DB: 对变化的文件创建新 TranslationJob
else event = installation_repositories
    W->>DB: 同步 Repository 表
end
```

1.2.3 在做什么（白话）

GitHub 在仓库发生事件（例如 **push**、**installation_repositories**）时，会向你配置的 URL **POST** 一段 **JSON**（Webhook）。本站必须：先验证请求真的来自 **GitHub**（HMAC 签名），再尽快返回 **200**（GitHub 若超时可能重试或认为失败），把原始载荷存进 `WebhookEvent` 表，由 **Worker** 异步慢慢处理——比对文件 hash、决定要不要新建翻译任务、或同步仓库列表。

参与者分工

角色	是什么	在本链路中的作用
GitHub	事件源	在 push / 安装变更时 POST Webhook。
middleware.ts	Next.js 边缘/中间层	验签；不通过直接 401，保护伪造请求。
`/api/webhooks/github`	API 路由	解析 payload、早返回、写入 `WebhookEvent`。
Neon	数据库	持久化事件，支持重试与审计。
Worker	异步消费者	读 `pending` 事件，调 GitHub Compare API，更新 DB，必要时创建 `TranslationJob`。

为什么要「**3** 秒内早返回」?
GitHub 对 Webhook 响应时间有要求（常见讨论阈值约 **10s**）。翻译分析可能很慢，若放在同步 HTTP 里容易超时；所以 HTTP 只负责收条 + 入库，重活交给 Worker。

分支：两种典型事件

事件类型	Worker 在做什么	业务含义
`push`	Compare API 看改了哪些文件 → 与已有 `FileTranslation.sourceHash` 比 → 变的才开新 Job	增量翻译：只处理变更，控制成本。
`installation_repositories`	更新 `Repository` 表	用户在 GitHub 上给 App 增删仓库访问时，本站列表与真实权限一致。

与 **1.2.2** 的关系
1.2.3 通常结束于「创建新的 **TranslationJob**」；真正翻译仍走 1.2.2 里那一套 Worker 处理 Job 的逻辑（复用同一套「出队 → 翻译 → 写回」能力）。

```
flowchart TB
    GH[GitHub POST Webhook]
    MW[middleware 校验签名]
    WH_Route["路由 /api/webhooks/github"]
    DB[(WebhookEvent 表)]
    W[Worker 异步处理]
    GH --> MW --> WH_Route
    WH_Route -->|INSERT pending| DB
    WH_Route -->|尽快 200| GH
    W -->|SELECT pending| DB
    W -->|必要时创建 Job| DB
```

1.3 模块分层（代码层面）

1.3.0 新手如何理解这一节
上图不是「部署拓扑」，而是代码应当怎么放、谁可以依赖谁：上层负责 HTTP/UI，中层负责领域逻辑（GitHub、翻译、队列），底层统一用 Prisma 访问数据库。这样单测、替换实现（例如将来换 VCS）都更容易。

四条链路在分层上的落点（与 **§1.2** 对照）

§1.2 链路	主要触碰的目录（概念上）	说明
1.2.1 登录 / 安装	app/（页面与 /api/auth）、lib/auth、lib/github、经 lib/db 写 Prisma	表现层发起，业务层封装 OAuth/App，数据层落库。
1.2.2 翻译 + PR	app/（Action / 轮询 API）、lib/translator、lib/queue、lib/github、lib/db	Worker 逻辑放在 lib/，由 app/api/worker 薄封装触发。
1.2.3 Webhook	middleware.ts、app/api/webhooks、lib/github、lib/queue（消费）、lib/db	验签尽量靠前；路由层尽快写 DB。
读模型 / 配置	lib/db + prisma/	Schema 与迁移是结构真相来源（见 §3.2 与当前仓库 prisma/schema.prisma 可能分阶段对齐）。

分层职责表

层	路径	放什么	不该放什么
表现层	app/	页面、布局、Route Handlers、middleware、Server Actions 的入口	不直接 new PrismaClient()；不写复杂 GitHub 调用细节
业务层	lib/*	按领域的纯逻辑与对外的 index.ts API	不直接绑定具体 React 组件
数据访问	lib/db.ts	Prisma 单例、（可选）事务辅助	业务规则应尽量回到 lib/*
元数据	prisma/	schema.prisma、migrations/	不在此写运行时业务代码

```
flowchart LR
    subgraph A["app/ 表现层"]
        A1["(auth)/*"]
        A2["(dashboard)/*"]
        A3["api/*"]
    end
    subgraph B["lib/ 业务层"]
        B1["lib/auth"]
        B2["lib/github"]
        B3["lib/translator"]
        B4["lib/queue"]
        B5["lib/vcs<br/>(P1 抽象)"]
```

```
end
subgraph C["lib/db.ts 数据层"]
    C1["PrismaClient 单例"]
end
subgraph D["prisma/ 元数据"]
    D1["schema.prisma"]
    D2["migrations/"]
end

A --> B
B --> C
C --> D
B2 -.P1 重构.-> B5
```

上图箭头 **A → B → C → D** 表示依赖方向：页面与 API 依赖业务模块，业务模块依赖数据库访问，数据库访问对应 Prisma 元数据。`lib/vcs` 虚线表示 **P1** 才把 GitHub 抽象成通用 VCS，当前 MVP 可仍以 `lib/github` 为主。

依赖规则（结合上图）

允许	禁止（强约束）
`app/*` ⟶ `lib/*/index`（业务入口）	`app/*` 直接 `import @prisma/client`
`lib/github` 等 ⟶ `lib/db`	`import lib/github/internal/...` 这类跨模块深路径（应用侧）
各 `lib` 子包对外只导出 `index.ts` 中声明的 API	在 `app` 里堆长串 Octokit 调用而不下沉到 `lib/github`

强约束：`app/` 不直接 import `@prisma/client`，必须经 `lib/db.ts`；`lib/` 各子模块只通过根 `index.ts` 对外暴露，禁止跨模块深路径引用。

一条请求的「纵向切片」示例（帮助把 **§1.2** 与 **§1.3** 连起来）
用户提交翻译表单：`app/(dashboard)/...` 里触发 Server Action ⟶ Action 调用 `lib/translator` 的 `createJob(...)` ⟶ 内部用 `lib/db` 事务写入 Job ⟶ 返回 `jobId` 给浏览器；随后 Cron 命中 `app/api/worker/tick` ⟶ 路由里调用 `lib/queue` / `lib/translator` 出队并执行 ⟶ 通过 `lib/github` 读写远程仓库。整条链路上，只有 **`lib/db`** 以下才碰 **Prisma**。

---

2. 技术选型

2.1 前端

领域	选型	理由	放弃的候选
框架	**Next.js 16 App Router**	前后端同构、Server Components 默认降低水合成本、Vercel 原生支持	Remix（Vercel 加持弱）、Vite+Express（两套部署麻烦）
语言	**TypeScript strict**	规则文件硬性要求；多人协作/AI 协作提示精度更高	原生 JS（对 AI 生成代码的可信度太低）
UI 组件	**shadcn/ui**	代码级复制而非 npm 依赖，可完全自定义；无障碍由底层 primitives 保障（**M0** 预设为 **Base UI**；历史方案亦常用 Radix）	MUI（主题学习成本高）、Chakra（未来维护不确定）
样式	**Tailwind CSS**	与 shadcn 绑定；utility-first 与 AI 生成搭配极佳	CSS Modules（冗长）、styled-components（已式微）
图标	**lucide-react**	shadcn 默认搭配，2000+ 图标全覆盖	heroicons（数量偏少）
表单	**react-hook-form + zod**	与后端 zod schema 共用类型，end-to-end type safe	formik（类型推导弱）
实时进度	短轮询（**2s**），GA 前再升级 SSE	Vercel Node Runtime `maxDuration` ≤ 60s，SSE 长连接撑不住翻译时长；短轮询成本可控、实现简单	SSE（阻于 serverless 时长限制）、WebSocket（双向过剩）
状态管理	**React Server Components + URL state** + 少量 **zustand**	服务端状态不落客户端；zustand 仅用于跨组件 UI 状态（Toast、Modal）	Redux（重）、Jotai（够用但 zustand 生态更大）
数据请求	**Server Actions** + 原生 **fetch**	Next.js 16 Server Actions 覆盖 90% 表单场景；RSC 里直接 await	SWR/React Query（RSC 下必要性下降）

2.2 后端（仍在 **Next.js** 内）

领域	选型	理由
Runtime	**Node.js runtime**（非 Edge）	Prisma 不完全支持 Edge；Octokit 依赖 Node crypto
API 风格	**Server Actions + REST (`/api/*`)**	Server Action 给内部表单；REST 给 Webhook/SSE/外部调用
入参校验	**zod**（规则强制）	运行时 + 编译时双重类型安全
GitHub SDK	**@octokit/app + @octokit/rest**	官方维护；`@octokit/app` 原生支持 GitHub App 鉴权（JWT + Installation Token）
Webhook 校验	**@octokit/webhooks**	内置 HMAC-SHA256 签名校验，防伪零成本
Markdown 解析	**remark + remark-parse + unified**	AST 操作成熟；可精确隔离代码块/行内代码/URL/图片
任务调度	**Vercel Cron + DB** 行级锁	`SELECT ... FOR UPDATE SKIP LOCKED` 天然防重
日志	**pino**（本地）+ **Vercel Logs**（线上）	结构化 JSON，便于未来接 Datadog/Logtail
错误上报	**@sentry/nextjs**（P1 接入）	Source map 上传+面包屑，性价比最高

2.3 数据库与 ORM

项	选型	理由
DBMS	**Neon Postgres**	Serverless、分支功能（dev/stage/prod 隔离）、免费额度够 MVP
ORM	**Prisma**	Schema 即文档；`prisma migrate` 迁移链路完整；TypeScript 类型自动生成
连接池	**Neon 内置 pooler（`-pooler` endpoint）**	无需自建 PgBouncer；Serverless 函数天然多连接
迁移策略	`prisma migrate dev`（本地）→ `prisma migrate deploy`（Vercel build hook）	零停机、版本化、可回滚
备份	Neon 自动快照（7 天，Free Tier）	MVP 阶段够用；GA 前升 Pro 改 30 天 PITR

2.4 第三方服务

服务	用途	成本策略	替代备案
GitHub App	OAuth 登录 + 仓库读写权限	免费	无（核心依赖）
OpenRouter	多模型统一接入	学习期全走 `:free` 后缀；GA 后允许用户自带 key	直连 OpenAI/Anthropic（放弃，失去路由灵活性）
Neon	Postgres	Free Tier；GA 前升 Launch Plan	Supabase（备案）
Vercel	部署	Hobby 起步；商用升 Pro	Cloudflare Pages（备案，但 Next.js 适配稍差）
cloudflared	本地 Webhook 调试	免费	ngrok（免费版 session 限制）
Stripe（P2）	订阅计费	按交易额抽成	Paddle（Merchant of Record，合规省心）
Sentry（P1）	错误监控	免费 5k events/月	Vercel 原生日志（够用但无告警）

2.5 工具链

项	选型
包管理	**pnpm**（规则强制，workspace 友好）
Lint	ESLint + `@typescript-eslint` + `eslint-plugin-import`（按分层强制依赖方向）
格式化	Prettier + `prettier-plugin-tailwindcss`
Git Hooks	Husky + lint-staged
提交规范	Conventional Commits + commitlint
测试	**Vitest**（单元）+ **Playwright**（E2E，P1）

3. 数据模型设计

3.1 ER 图

```
erDiagram
    User ||--o{ Installation : owns
    User ||--o{ Session : has
    Installation ||--o{ Repository : includes
    Repository ||--o{ TranslationJob : triggers
    Repository ||--o{ RepoConfig : configures
    TranslationJob ||--o{ FileTranslation : contains
    TranslationJob }o--|| AiModel : uses
    Repository ||--o{ Glossary : owns
    Repository ||--o{ WebhookEvent : receives
    FileTranslation }o--o{ TranslationMemory : contributes

    User {
        string id PK
        int githubId UK
        string login
        string name
        string avatarUrl
        string accessTokenEnc "OAuth token (加密)"
        datetime createdAt
        datetime updatedAt
    }
    Session {
        string id PK
        string userId FK
        string tokenHash UK
        datetime expiresAt
    }
    Installation {
        string id PK
```

```
    int githubInstallationId UK
    string userId FK
    string accountLogin
    string accountType "User|Organization"
    datetime suspendedAt "可空"
    datetime createdAt
}
Repository {
    string id PK
    int githubRepoId UK
    string installationId FK
    string fullName "仅展示，不做唯一"
    string defaultBranch
    bool private
    bool isEnabled "是否开启自动增量"
    datetime createdAt
}
RepoConfig {
    string id PK
    string repositoryId UK
    string[] defaultTargetLangs "用户偏好语言"
    json ignoreGlobs "glob 规则 JSON 数组"
    string glossaryPath ".github/i18n.yaml"
    string branchStrategy "append|new-each"
    int dailyCharLimit
}
TranslationJob {
    string id PK
    string repositoryId FK
    string userId FK "触发者"
    string status "pending|running|succeeded|failed|cancelled"
    string trigger "manual|webhook|retry"
    string commitSha "对应的原文 commit"
    string[] targetLangs
    string modelId
    string branchName "i18n/zh-CN-2026xxxx"
    string prUrl
    int totalFiles
    int doneFiles
    int failedFiles
    string errorMessage
    datetime queuedAt
    datetime startedAt
    datetime finishedAt
}
FileTranslation {
    string id PK
    string jobId FK
    string repositoryId FK
```

```
    string path "README.md"
    string lang "zh-CN"
    string sourceHash "原文 SHA-256"
    string translatedHash "译文 SHA-256"
    int tokensIn
    int tokensOut
    int durationMs
    string status "queued|running|succeeded|failed|skipped"
    string skipReason "hash-match|ignored|oversized"
    string errorMessage
    datetime startedAt
    datetime finishedAt
}
AiModel {
    string id PK
    string displayName
    bool isFree
    int contextWindow
    decimal pricePerMTokenIn
    decimal pricePerMTokenOut
    bool enabled
}
Glossary {
    string id PK
    string repositoryId FK
    string source "Agent"
    string lang "zh-CN"
    string target "智能体"
    string scope "repo|global"
}
WebhookEvent {
    string id PK
    string deliveryId UK "X-GitHub-Delivery"
    string eventType "push|installation|installation_repositories"
    string installationId FK "可空"
    json payload "压缩后的原始载荷"
    string status "pending|processed|failed"
    string errorMessage
    datetime receivedAt
    datetime processedAt
}
TranslationMemory {
    string id PK
    string repositoryId FK "可空=全局 TM"
    string sourceLang
    string targetLang
    string sourceText
    string targetText
    string sourceHash UK "sha256(source)"
```

```
        int hitCount
        datetime lastUsedAt
        datetime createdAt
    }
```

3.2 Prisma Schema（可直接落地）

```
// prisma/schema.prisma
generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider   = "postgresql"
  url        = env("DATABASE_URL")
  directUrl  = env("DATABASE_URL_UNPOOLED") // Neon migration 用非池化连接
  extensions = [pgcrypto, citext]
}

// ---------- 认证 ----------
model User {
  id              String         @id @default(cuid())
  githubId        Int            @unique
  login           String
  name            String?
  email           String?        @db.Citext
  avatarUrl       String?
  accessTokenEnc  String         // AES-256-GCM 加密后的 OAuth token
  createdAt       DateTime       @default(now())
  updatedAt       DateTime       @updatedAt

  sessions        Session[]
  installations   Installation[]
  jobs            TranslationJob[]

  @@index([login])
}

model Session {
  id        String   @id @default(cuid())
  userId    String
  tokenHash String   @unique           // 存 sha256(opaque token), cookie 里是原文
  expiresAt DateTime
  createdAt DateTime @default(now())
  user      User     @relation(fields: [userId], references: [id], onDelete:
Cascade)

  @@index([userId])
}
```

```
// ---------- GitHub App ----------
model Installation {
  id                    String       @id @default(cuid())
  githubInstallationId  Int          @unique
  userId                String
  accountLogin          String
  accountType           String       // "User" | "Organization"
  suspendedAt           DateTime?
  createdAt             DateTime     @default(now())
  updatedAt             DateTime     @updatedAt

  user         User         @relation(fields: [userId], references: [id],
onDelete: Cascade)
  repositories Repository[]

  @@index([userId])
}

model Repository {
  id             String       @id @default(cuid())
  githubRepoId   Int          @unique            // GitHub 侧全局唯一, transfer 后不
变
  installationId String
  fullName       String                          // "owner/repo", 展示用, 不作唯一键
(repo 可 transfer)
  defaultBranch  String       @default("main")
  private        Boolean      @default(false)
  isEnabled      Boolean      @default(false)    // 是否订阅增量翻译 (运行态开关, 保留
在主表)
  createdAt      DateTime     @default(now())
  updatedAt      DateTime     @updatedAt

  installation  Installation     @relation(fields: [installationId], references:
[id], onDelete: Cascade)
  jobs          TranslationJob[]
  files         FileTranslation[]
  config        RepoConfig?
  glossary      Glossary[]
  memories      TranslationMemory[]

  @@index([installationId])
  @@index([fullName])                            // 搜索 / 展示用
}

model RepoConfig {
  id                   String   @id @default(cuid())
  repositoryId         String   @unique
  defaultTargetLangs   String[] @default([])              // 用户偏好的目标语言集合
```

```
(从 Repository 迁入)
  ignoreGlobs          Json     @default("[]")
  glossaryPath         String   @default(".github/i18n.yaml")
  branchStrategy       String   @default("append")       // "append" | "new-each"
  dailyCharLimit       Int      @default(500000)
  updatedAt            DateTime @updatedAt

  repository Repository @relation(fields: [repositoryId], references: [id],
onDelete: Cascade)
}

// ---------- 翻译核心 ----------
model TranslationJob {
  id             String    @id @default(cuid())
  repositoryId   String
  userId         String
  status         String    @default("pending")   // pending|running|succeeded|
succeeded_with_errors|failed|cancelled
  trigger        String    @default("manual")    // manual|webhook|retry
  commitSha      String?                          // 触发时的 HEAD SHA
  targetLangs    String[]
  modelId        String
  branchName     String?
  prUrl          String?
  totalFiles     Int       @default(0)
  doneFiles      Int       @default(0)
  failedFiles    Int       @default(0)
  errorMessage   String?
  queuedAt       DateTime  @default(now())
  startedAt      DateTime?
  finishedAt     DateTime?

  repository Repository        @relation(fields: [repositoryId], references:
[id], onDelete: Cascade)
  user       User              @relation(fields: [userId], references: [id])
  files      FileTranslation[]

  @@index([status, queuedAt])       // worker 拉任务用
  @@index([repositoryId, queuedAt])
}

model FileTranslation {
  id              String   @id @default(cuid())
  jobId           String
  repositoryId    String
  path            String
  lang            String
  sourceHash      String                       // sha256(原文)
  translatedHash  String?                      // sha256(译文)
```

```
  tokensIn        Int      @default(0)
  tokensOut       Int      @default(0)
  durationMs      Int      @default(0)
  status          String   @default("queued")  // queued|running|succeeded|
failed|skipped
  skipReason      String?                      // hash-match|ignored|oversized
  errorMessage    String?
  startedAt       DateTime?
  finishedAt      DateTime?
  createdAt       DateTime @default(now())

  job         TranslationJob @relation(fields: [jobId], references: [id],
onDelete: Cascade)
  repository  Repository     @relation(fields: [repositoryId], references: [id],
onDelete: Cascade)

  // 同一仓库/语言/路径, 最新的 sourceHash 唯一 → 增量判断
  @@index([repositoryId, path, lang, createdAt(sort: Desc)])
  // 增量翻译热路径: 给定 (repo, path, lang, newHash), EXISTS 命中 O(1) 跳过
  @@index([repositoryId, path, lang, sourceHash])
  @@index([jobId, status])
}

model Model {
  id                 String   @id                // "z-ai/glm-4.5-air:free"
  displayName        String
  isFree             Boolean  @default(false)
  contextWindow      Int
  pricePerMTokenIn   Decimal? @db.Decimal(10, 4)
  pricePerMTokenOut  Decimal? @db.Decimal(10, 4)
  enabled            Boolean  @default(true)
  updatedAt          DateTime @updatedAt
}

// ---------- 配置与长期资产 ----------
model Glossary {
  id           String   @id @default(cuid())
  repositoryId String
  source       String
  lang         String
  target       String
  scope        String   @default("repo")   // repo|global
  createdAt    DateTime @default(now())

  repository Repository @relation(fields: [repositoryId], references: [id],
onDelete: Cascade)

  @@unique([repositoryId, source, lang])
}
```

```
model TranslationMemory {
  id           String   @id @default(cuid())
  repositoryId String?                        // null = 全局 TM
  sourceLang   String
  targetLang   String
  sourceText   String   @db.Text
  targetText   String   @db.Text
  sourceHash   String   @unique                // sha256(sourceText + sourceLang
+ targetLang)
  hitCount     Int      @default(0)
  lastUsedAt   DateTime @default(now())
  createdAt    DateTime @default(now())

  repository Repository? @relation(fields: [repositoryId], references: [id],
onDelete: SetNull)

  @@index([sourceLang, targetLang])
}

// ---------- Webhook 审计 ----------
model WebhookEvent {
  id             String   @id @default(cuid())
  deliveryId     String   @unique             // X-GitHub-Delivery (GitHub 保证全局
唯一 → 天然幂等 key)
  eventType      String                        // push|installation|
installation_repositories|...
  installationId String?
  payload        Json                          // gzip 前先 base64？→ MVP 直存 json
省事
  status         String   @default("pending")  // pending|processed|failed|
ignored
  errorMessage   String?
  receivedAt     DateTime @default(now())
  processedAt    DateTime?

  @@index([status, receivedAt])
}
```

3.3 关键字段设计解释（为什么这么写）

字段	决策	原因
`User.accessTokenEnc`	AES-256-GCM 对称加密存	OAuth token 一旦泄露直接等于用户账号被盗；对称加密 + `ENCRYPTION_KEY` 环境变量分离是成本最低的方案
`Session.tokenHash`	存哈希不存明文	即使数据库脱库，攻击者拿到 hash 也无法登录；cookie 里存原 token
`FileTranslation.sourceHash`	SHA-256(原文)	F-13 增量判断核心：同一 (`repositoryId`, `path`, `lang`) 若新 sourceHash 与最新一条相同 → 跳过
`WebhookEvent.deliveryId` `UNIQUE`	用 GitHub 的 X-GitHub-Delivery 做幂等键	GitHub 会重复投递（超时会重试），UNIQUE 约束让重复事件自动被 DB 层拒绝
`TranslationJob.status` 字符串字面量	不用 enum	规则 `code-style.mdc` 第 13 行；迁移灵活，Prisma + TS 联合类型仍保留类型安全
`TranslationMemory.sourceHash` `UNIQUE`	同文本命中 O(1)	TM 查询是翻译热路径，索引必须极简
`Repository.installationId` 而不是 `userId`	跟着 GitHub 的权限模型走	一个仓库的操作权依赖 Installation Token 而非用户 token；Installation 被吊销时整个仓库自动失效

---

4. 核心 API 接口设计

4.1 接口约定

- **Base URL**：`https://<domain>/api`
- 认证：Cookie `session` → 中间件解出 `userId` 注入 `request.ctx`
- 内容类型：`application/json`（Webhook 除外，原样保留）
- 入参校验：全部用 zod schema，校验失败返回 422
- 错误格式：

```
{ "error": { "code": "REPO_NOT_FOUND", "message": "...", "details": {} } }
```

- 幂等：所有 `POST` 接受可选 `Idempotency-Key` header，24h 内同 key 返回相同结果

4.2 认证域

Method	Path	入参	返回	说明
GET	/api/auth/github/start	?redirect=/dashboard	302	发起 OAuth，state 写进 httpOnly cookie 防 CSRF
GET	/api/auth/github/callback	?code&state	302	换 token、建会话、回跳
POST	/api/auth/logout	-	204	清 Session 表 + 过期 cookie
GET	/api/auth/me	-	{ user }	当前登录用户

4.3 仓库域

Method	Path	入参	返回	说明
GET	/api/repos	?installationId&page&size	{ items, nextCursor }	列出当前用户授权的仓库（源自本地 DB，定期从 GitHub 同步）
POST	/api/repos/sync	{ installationId }	{ synced: number }	手动触发从 GitHub 拉取最新仓库列表
GET	/api/repos/:id	-	{ repo, config, glossaryCount }	单仓库详情
PATCH	/api/repos/:id	{ isEnabled?, targetLangs?, config? }	{ repo }	更新订阅状态 / 目标语言 / 配置
GET	/api/repos/:id/files	?ref	{ files: [{ path, size }] }	预览仓库里所有 .md 文件（调用 GitHub Tree API）

4.4 翻译任务域

Method	Path	入参（zod）	返回	说明
POST	`/api/jobs`	`{ repositoryId, targetLangs[], modelId?, pathGlob? }`	`{ jobId, totalFiles }`	创建手动任务，仅写 **DB**，worker 异步处理
GET	`/api/jobs/:id`	-	`{ job, progress, files: [{path, lang, status}] }`	快照查询，前端每 2s 轮询，任务终态（succeeded/failed/cancelled）后停止
~~GET `/api/jobs/:id/stream`~~	-	-	-	**MVP** 不做：Node Runtime `maxDuration` 限制下 SSE 会断，改用短轮询；GA 再评估
GET	`/api/jobs/:id/files`	`?status&lang`	`{ items }`	细粒度查看每个文件结果
POST	`/api/jobs/:id/cancel`	-	`{ job }`	标记 cancelled；已在跑的文件跑完即停
POST	`/api/jobs/:id/retry`	`{ onlyFailed?: boolean }`	`{ newJobId }`	基于原任务配置创建重试任务

4.5 术语表域

Method	Path	入参	说明
GET	`/api/repos/:id/glossary`	`?lang`	列表
POST	`/api/repos/:id/glossary`	`{ entries: [{ source, lang, target }] }`	批量新增/覆盖
DELETE	`/api/repos/:id/glossary/:entryId`	-	删除单条
POST	`/api/repos/:id/glossary/sync-from-file`	`{ path: ".github/i18n.yaml" }`	从仓库 YAML 拉取覆盖（P1）

4.6 Webhook 域

Method	Path	说明
POST	`/api/webhooks/github`	GitHub 所有事件入口，必须经 `middleware.ts` 做签名校验；**3** 秒内返回 **200**，真实逻辑走 WebhookEvent 异步处理

处理的事件白名单：

事件	处理逻辑
`installation.created`	落 Installation + 同步 repositories
`installation.deleted`	级联软删（保留历史记录，但 Installation 标 `suspendedAt`）
`installation.suspend` / `unsuspend`	更新 `suspendedAt`
`installation_repositories.added/removed`	同步 Repository 表
`push`（默认分支）	若 `Repository.isEnabled = true` → 创建 webhook 触发的 TranslationJob
其他	忽略

4.7 Worker & 内部域

Method	Path	保护方式	说明
POST	`/api/worker/tick`	`x-cron-secret` header 校验	Vercel Cron 每分钟调一次；拉 pending 任务跑；单次执行 ≤ **50s**（防 Vercel 60s 超时）
POST	`/api/worker/webhooks/drain`	同上	消费 `WebhookEvent`（隔离于翻译 worker 避免互相阻塞）

4.8 Server Actions（表单优先场景）

放在 `app/(dashboard)/_actions/*.ts`，不走 `/api`：

- `createTranslationJobAction(input)` ⟶ 表单"开始翻译"按钮
- `updateRepoConfigAction(input)` ⟶ 仓库设置页
- `upsertGlossaryAction(input)` ⟶ 术语表编辑

为什么拆：Server Action 带 progressive enhancement（无 JS 也能用），表单场景首选；SSE / 跨端调用 / Webhook 只能走 REST。

4.9 zod schema 示例（createJob）

```
// app/api/jobs/route.ts
import { z } from "zod";

export const createJobSchema = z.object({
  repositoryId: z.string().cuid(),
  targetLangs: z.array(z.enum(["zh-CN", "ja", "es", "fr", "de",
"ru"])).min(1).max(7),
  modelId: z.string().optional(),          // 不传走 .env 默认
  pathGlob: z.string().optional(),          // 例: "docs/**/*.md"
});

export type CreateJobInput = z.infer<typeof createJobSchema>;
```

---

5. 核心算法 & 关键实现决策

5.1 Markdown 保护式翻译（F-05 / F-16）

```
flowchart LR
    A[原文 .md] --> B[remark parse → AST]
    B --> C["遍历 AST: <br/>代码块/行内代码/URL/图片 → 占位符 P1"]
    C --> D[抽出纯文本片段]
    D --> E["分段<br/>(按标题与段落, 约 context 上限的 60%)"]
    E --> F[逐段调 OpenRouter]
```

```
F --> G[译文回填占位符]
G --> H[remark stringify → 新 .md]
H --> I{AST 对齐校验}
I -- 通过 --> J[写入仓库]
I -- 失败 --> K[本文件回滚，记录 errorMessage]
```

关键点：- 占位符 `{{P1}}/{{P2}}` 比“把代码块塞给模型并祈祷它别改”可靠 100 倍 - AST 对齐校验：原文/译文 AST 的“非文本节点骨架”必须完全一致（标题级别、列表结构、代码块数量），否则判定破坏 - 分段时保留上一段末尾 **3** 句 + 下一段开头 **2** 句作为上下文窗口，避免专有名词翻译漂移

PR 产物策略（部分失败场景）：- 只要本次任务 ≥ 1 个文件翻译成功，就创建/更新 PR；完全失败（0 成功）则不建 PR，任务标 `failed` - PR 描述里用一张表格列出每个文件的状态：✅ 已翻译 / ⏭️ 跳过(hash未变) / ❌ 失败(原因) - 失败的文件在表格行尾附 [重试] 链接 → 深链到 `/dashboard/jobs/:id?retry=<filePath>`，一键创建 onlyFailed 重试任务（对应 §4.4 的 `POST /api/jobs/:id/retry`）- `TranslationJob.failedFiles > 0` 时 `status=succeeded_with_errors`（新增状态值），UI 在任务卡片上以黄色徽章警示

5.2 并发 & 限流

场景	策略
单个 worker tick 内并行翻译文件	`Promise.allSettled + p-limit(5)`（避免 OpenRouter 瞬时打爆）
单仓库日字符数上限	`RepoConfig.dailyCharLimit`，超限任务直接 `status=failed, errorMessage='daily_limit_exceeded'`
OpenRouter 429	指数退避 3 次（1s/3s/9s）；仍失败换备用模型；仍失败标 file failed
任务级锁	不锁，worker 用 `FOR UPDATE SKIP LOCKED` 保证同一 job 不被两个 tick 同时拉起

5.3 增量翻译核心 SQL（F-13）

```sql
-- 给定 (repositoryId, path, lang, newSourceHash)，直接探测是否存在已成功的同哈希记录
SELECT EXISTS (
  SELECT 1 FROM "FileTranslation"
  WHERE "repositoryId" = $1
    AND "path" = $2
    AND "lang" = $3
    AND "sourceHash" = $4
    AND "status" = 'succeeded'
) AS hit;
-- 命中则跳过翻译 (status='skipped', skipReason='hash-match')
```

走 `@@index([repositoryId, path, lang, sourceHash])` 复合索引，命中 O(1)；比“取最新一条再比对”更稳：即使历史上曾翻译失败过，新 hash 也能正确决策。

6. 安全设计

6.1 认证与会话

项	方案
会话载体	httpOnly + Secure + SameSite=Lax cookie；`token` = 32 字节随机 ⟶ `Session.tokenHash = sha256(token)`
有效期	滑动 7 天，敏感操作（如删除仓库）需 `within_30min` 二次确认
CSRF	Server Actions 走 Next.js 原生保护；REST POST 对同源限制 + `Origin` 校验
XSS	React 自带转义；禁止 `dangerouslySetInnerHTML`；Markdown 渲染用 `react-markdown` + `rehype-sanitize`

6.2 密钥管理

类别	存储
GitHub App 私钥 (`.pem`)	Vercel 环境变量 `GITHUB_APP_PRIVATE_KEY`（base64 编码后塞一行），本地放 `D:\hyd001\secrets\` 外部目录
OAuth Client Secret	`GITHUB_OAUTH_CLIENT_SECRET` 环境变量
Webhook Secret	`GITHUB_WEBHOOK_SECRET`；用 `@octokit/webhooks` 校验
数据库 URL	`DATABASE_URL`（pooled）+ `DATABASE_URL_UNPOOLED`（migration）
OpenRouter API Key	`OPENROUTER_API_KEY`
对称加密主密钥	`ENCRYPTION_KEY`（32 字节 base64），用于加密 `User.accessTokenEnc`
Cron 调用密钥	`CRON_SECRET`，`/api/worker/*` 路由强校验

硬约束：- `.env.local` 永不进 git（`.gitignore` 已覆盖）- `.env.example` 只写 key 名 + 示例格式进 git - `lib/env.ts` 启动时用 zod 校验所有必需项，缺失直接 crash

6.3 Webhook 安全

- `middleware.ts` 对 `/api/webhooks/github` 唯一一条请求放行前校验 `X-Hub-Signature-256`（HMAC-SHA256）
- 未通过校验直接 401，不落库、不写日志正文

- `X-GitHub-Delivery` UNIQUE 防重放

6.4 权限校验矩阵（授权模型）

每个业务操作的权限检查点：

操作	校验
查看仓库详情	`Repository.installation.userId === ctx.userId`
创建翻译任务	同上 + `Installation.suspendedAt IS NULL` + **`Repository.private === false`**（MVP 仅公开仓库；前端过滤不可信，后端必须拒绝）
查看 Job	`Job.repository.installation.userId === ctx.userId`
Webhook 处理	不校验用户（GitHub 身份）→ 用签名校验 + installation_id 匹配

封装在 `lib/auth/guards.ts`，每个 handler 第一行调用，避免散落各处。

6.5 数据合规

项	措施
数据最小化	MVP 保留原文/译文用于调试；GA 前改为仅存 hash + 统计（对齐 PRD 6.1）
GDPR 数据删除	提供 `/dashboard/settings/delete-account` 入口，级联删除 User → Session / Installation / Job 的 `userId` 置空（保留审计需要的元数据匿名化）
私有仓库	MVP 仅允许公开仓库（PRD 开放问题倾向方案），前端在勾选时过滤 `private=true`
日志脱敏	pino 配置 `redact: ['req.headers.authorization', 'req.cookies', '*.accessToken*']`

---

7. 部署方案

7.1 环境拓扑

```
flowchart LR
    Dev["本地开发<br/>localhost:3000"] -->|cloudflared tunnel|
WebhookURL["https://dev-tunnel.<br/>trycloudflare.com"]
    Dev --> NeonDev[("Neon branch:<br/>dev")]
```

```
    Preview["Vercel Preview<br/>(每次 PR 自动)"] --> NeonPreview[("Neon branch<br/>preview + commit 短 SHA")]

    Prod["Vercel Production<br/>github-global.app"] --> NeonProd[("Neon branch:<br/>main")]

    GH["GitHub App"] -.Webhook.-> WebhookURL
    GH -.Webhook.-> Prod
```

- **Neon** 分支策略：每个 Vercel Preview 自动创建一个 Neon branch（通过 Vercel ↔ Neon integration），代码合并主干后 branch 自动销毁 → **Preview** 环境永远有独立数据库
- **GitHub App** 双开：`GitHub Global Dev` + `GitHub Global Prod` 两个 App，避免开发 webhook 污染生产数据

7.2 CI/CD

```
flowchart LR
    PR["PR 推送"] --> Lint["eslint + tsc --noEmit"]
    Lint --> Test["vitest 单测"]
    Test --> Build["next build + prisma generate"]
    Build --> Preview["Vercel Preview 部署"]
    Preview --> Review["人工 Review"]
    Review --> Merge["合并到 main"]
    Merge --> Migrate["prisma migrate deploy, Vercel build hook"]
    Migrate --> Deploy["Vercel Production 部署"]
    Deploy --> Smoke["E2E smoke, Playwright P1"]
```

上图节点文案已全部用引号包裹，避免 **`--noEmit`** 里的 **`--`** 被解析成流程图边语法（未加引号时会导致 GitHub 上整图无法渲染）。`next build` 与 `prisma generate` 在 Vercel 上通常同一次 build 内顺序执行。

7.3 环境变量清单（`.env.example`）

```
# --- 数据库 ---
DATABASE_URL=postgresql://user:pwd@ep-xxx-pooler.neon.tech/db?sslmode=require
DATABASE_URL_UNPOOLED=postgresql://user:pwd@ep-xxx.neon.tech/db?sslmode=require

# --- GitHub App ---
GITHUB_APP_ID=1439734
GITHUB_APP_CLIENT_ID=Iv23liZuUcTmF6FFrSck
GITHUB_APP_CLIENT_SECRET=xxxxxxxxxxxx
GITHUB_APP_PRIVATE_KEY_BASE64=LS0tLS1CRUdJTi...
GITHUB_APP_WEBHOOK_SECRET=xxxxxxxxxxxx

# --- OpenRouter ---
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxx
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
TRANSLATION_MODEL_PRIMARY=z-ai/glm-4.5-air:free
TRANSLATION_MODEL_FALLBACK=qwen/qwen-2.5-72b-instruct:free
```

```
# --- 会话 / 加密 ---
ENCRYPTION_KEY=base64:xxxxxxxxxxxx   # 32 bytes
SESSION_SECRET=base64:xxxxxxxxxxxx

# --- Worker ---
CRON_SECRET=xxxxxxxxxxxx

# --- App ---
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

7.4 Vercel 配置

- **Runtime**：所有路由默认 Node.js；只有 `app/(marketing)/` 静态页用 Edge
- 函数超时：`/api/worker/tick` 设 `maxDuration: 60`；其它默认 10s
- **Cron 触发方式（MVP）**：不使用 **Vercel Cron**，改由 **GitHub Actions** 定时任务 每分钟 `curl` 两个 worker 端点
  - 理由：Vercel Hobby 免费版 Cron 仅支持 `daily` 级别，`* * * * *` 需升级 Pro（$20/月）；GitHub Actions 在公开仓库免费、频率不限
  - GA 后如需更可靠调度，再评估升级 Vercel Pro 或迁移到 Upstash QStash
- **Worker 端点保护**：`x-cron-secret: $CRON_SECRET` 必须匹配，否则 401
- **GitHub Actions 配置（`.github/workflows/worker-tick.yml`）**：

```yaml
name: Worker Tick
on:
  schedule:
    - cron: "* * * * *"   # 每分钟
  workflow_dispatch:
concurrency:
  group: worker-tick
  cancel-in-progress: false
jobs:
  tick:
    runs-on: ubuntu-latest
    steps:
      - name: Tick translation worker
        run: |
          curl -fsS -X POST "$APP_URL/api/worker/tick" \
            -H "x-cron-secret: $CRON_SECRET" || true
          curl -fsS -X POST "$APP_URL/api/worker/webhooks/drain" \
            -H "x-cron-secret: $CRON_SECRET" || true
        env:
          APP_URL: ${{ secrets.APP_URL }}
          CRON_SECRET: ${{ secrets.CRON_SECRET }}
```

注：GitHub Actions 的 cron 最小粒度是分钟，且在高峰时段偶有延迟（最坏 5~15 分钟），MVP 可接受。

7.5 凭据管理矩阵

三环境凭据严格隔离，避免“开发误写生产”的事故：

环境	GitHub App	Neon Postgres	Webhook 隧道	变量存储
本地开发	`GitHub Global Dev` App	Neon branch `dev`	`cloudflared tunnel` 临时域名	`.env.local`（`.gitignore` 屏蔽）
Vercel Preview	同 Dev App	Neon branch `preview-<sha>`（Vercel-Neon integration 自动创建/销毁）	Vercel Preview 域名	Vercel Preview Environment Variables
Vercel Production	`GitHub Global Prod` App	Neon branch `main`	自定义域名（如 `github-global.app`）	Vercel Production Environment Variables

硬约束：- 两个 GitHub App（Dev / Prod）分别有独立的 `APP_ID`、`CLIENT_ID`、`CLIENT_SECRET`、`PRIVATE_KEY`、`WEBHOOK_SECRET`，不共用 - `CRON_SECRET`、`ENCRYPTION_KEY` 三环境各自独立生成，不跨环境复用 - Vercel 环境变量按 Environment 维度（Development / Preview / Production）分别配置 - `.pem` 私钥原文只在本地 `D:\hyd001\secrets\` 留底；进 Vercel 前用 `base64 -w0` 压成单行塞 `GITHUB_APP_PRIVATE_KEY_BASE64`

7.6 可观测性

层面	方案	阶段
请求日志	Vercel Logs（默认）	MVP
结构化日志	pino → stdout → Vercel Logs	MVP
错误监控	Sentry `@sentry/nextjs`	P1
业务指标	`TranslationJob` / `FileTranslation` 直接 SQL 查询 → Grafana Cloud（免费方案）	P1
告警	Sentry 告警邮件 + Vercel 部署失败 webhook	P1
审计	`WebhookEvent` + `TranslationJob` 表即审计日志	MVP

---

8. 目录结构约定（照这个骨架开工）

```
Document translation/
├── app/
│   ├── (marketing)/              # 未登录公开页（静态，Edge）
│   │   ├── page.tsx              # 首页
│   │   └── pricing/page.tsx
│   ├── (auth)/
│   │   └── login/page.tsx
│   ├── (dashboard)/              # 需要登录，走 middleware 保护
│   │   ├── layout.tsx
│   │   ├── repos/
│   │   │   ├── page.tsx
│   │   │   └── [id]/
│   │   │       ├── page.tsx
│   │   │       └── settings/page.tsx
│   │   ├── jobs/
│   │   │   └── [id]/page.tsx
│   │   └── _actions/             # Server Actions
│   ├── api/
│   │   ├── auth/
│   │   │   └── github/
│   │   │       ├── start/route.ts
│   │   │       └── callback/route.ts
│   │   ├── repos/
│   │   ├── jobs/
│   │   ├── webhooks/github/route.ts
│   │   └── worker/
│   │       ├── tick/route.ts
│   │       └── webhooks/drain/route.ts
│   └── layout.tsx
├── middleware.ts                 # ⚠ 根目录，和 app/ 平级；会话注入 + webhook 签名校
验
├── lib/
│   ├── env.ts                    # zod 校验所有环境变量
│   ├── db.ts                     # PrismaClient 单例
│   ├── auth/
│   │   ├── index.ts
│   │   ├── oauth.ts
│   │   ├── session.ts
│   │   └── guards.ts
│   ├── github/
│   │   ├── index.ts
│   │   ├── app-client.ts         # @octokit/app 封装
│   │   ├── installation.ts
│   │   ├── repo.ts               # 列文件、读文件、创建分支/PR
│   │   └── webhooks.ts
│   ├── translator/
```

```
│   │   ├── index.ts
│   │   ├── openrouter.ts
│   │   ├── markdown-protect.ts   # remark AST 保护
│   │   ├── chunk.ts              # 分段策略
│   │   ├── prompt.ts
│   │   └── validator.ts          # AST 对齐校验
│   ├── queue/
│   │   ├── index.ts
│   │   ├── enqueue.ts
│   │   ├── worker.ts
│   │   └── webhook-consumer.ts
│   └── vcs/                      # P1: GitHub/GitLab/Gitee 适配器抽象
├── components/
│   ├── ui/                       # shadcn 生成
│   ├── repos/
│   ├── jobs/
│   └── shared/
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── docs/
│   ├── PRD.md
│   ├── TechDesign.md             # 本文件
│   └── handoff-*.md
├── tests/
│   ├── unit/
│   └── e2e/                      # P1
├── .cursor/rules/
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json
├── next.config.mjs
├── tailwind.config.ts
├── vercel.json
└── README.md
```

---

9. 实施路线（对齐 PRD 里程碑）

阶段	对齐 PRD	本文档中对应的章节	技术交付物
M0 脚手架	第 1 周	§2 + §7 + §8	Next.js **16** init、Prisma **6** init、Tailwind **v4**、shadcn init、env 校验、`lib/db.ts`、CI 跑通
M1 核心链路	第 2–3 周	§3 (User/Installation/Repository) + §4.2 + §4.3	OAuth 登录跑通、GitHub App 安装回调、仓库列表展示
M2 MVP	第 4–6 周	§3 (TranslationJob/FileTranslation) + §4.4 + §4.7 + §5.1	手动触发翻译、worker 闭环、PR 创建
M3 Beta	第 7 周	§7.1 + §7.2	生产部署、Prod App 创建、20 位种子用户
M4 增量	第 8–10 周	§4.6 + §5.3	Webhook + hash 比对 + 术语表
M5 付费化	第 11–14 周	（本文档未展开，后续 DesignDoc 增补）	Stripe、用量看板

每个阶段完工时，本文档对应章节打勾；如有变更，在章节末尾追加“v1.x 变更说明”小节，不删旧内容。

---

10. 决策记录（已锁定，v1.1 生效）

编号	问题	✅ 决定	影响范围
TD-01	MVP 是否允许私有仓库？	否，仅公开仓库。后端创建任务时强制校验 `Repository.private === false`，前端仅做 UX 过滤	§6.4 授权矩阵、§6.5 合规
TD-02	术语表存储位置？	仓库内 **`.github/i18n.yaml`** + 平台 **DB** 双向同步，仓库 YAML 为权威来源，平台 UI 作编辑入口	§4.5 + 新增 `/api/repos/:id/glossary/sync-from-file`
TD-03	翻译 PR 同分支追加还是每次新开？	默认追加（`RepoConfig.branchStrategy = "append"` 为默认）；`new-each` 保留为高级选项，UI 暂不暴露	§5.1 产物链路
TD-04	是否在 M2 就引入 Sentry？	否，**M3 Beta** 再接。M0/M1/M2 使用 Vercel Logs + pino 结构化日志兜底	§7.6 可观测性节奏
TD-05	Worker 是否拆独立 Vercel 项目？	否，**MVP** 同项目内通过 **GitHub Actions** 定时 **cron** 触发；GA 前再评估迁移到 Inngest / QStash 并拆独立 Worker 项目	§1.1 架构演进、§7.4 Cron 方案
TD-06	Worker 定时触发方式	**GitHub Actions `* * * * *` cron → `curl /api/worker/tick` 与 `/api/worker/webhooks/drain`**（Vercel Hobby 不支持每分钟 cron，升 Pro 再切回）	§7.4
TD-07	翻译进度前端实时性	**MVP** 前端 **2s** 短轮询 `/api/jobs/:id`；SSE 推到 GA 再评估（Serverless `maxDuration` 限制使 SSE 长连接不可靠）	§2.1 前端、§4.4 API
TD-08	免费翻译模型选型	主模型 **`z-ai/glm-4.5-air:free`**，备 **`qwen/qwen3-next-80b-a3b-instruct:free`**、**`openai/gpt-`**	§2.4、`.env.example`

---

11. 文档维护

- 本文件 = 施工蓝图。任何技术选型 / 数据模型变更必须先改本文件、再改代码，通过 PR review 把关
- 与 PRD 的分工：PRD 讲“为什么做、做什么”，本文件讲“怎么做、用什么做”
- 大版本变更（v1 → v2）时保留旧版本在 `docs/archive/TechDesign-v1.md`

v1.2 变更说明（**2026-04-22**，**M0** 脚手架实测对齐）

项	文档原表述（v1.1）	仓库实际（以 `package.json` 为准）	处理
Next.js	多处写 **15**	**16.2.4**（`create-next-app@latest` 默认）	全文统一为 **16**；架构图 subgraph 同步
React / Tailwind	未钉死小版本	**React 19.2.x**、**Tailwind CSS v4**（PostCSS 插件链路）	§9 M0 交付物补充
Prisma	§3.2 示例为通用写法	**Prisma 6.19.x**；不跟 Prisma 7（`schema.prisma` 内不再写 `url` / `directUrl`）	M0~M2 锁 6.x；升 7 需另开设计变更
shadcn	Radix 表述为主	当前为 **base-nova** + **`@base-ui/react`**（CLI 默认预设）	技术栈仍以「shadcn/ui 生态」表述；组件实现以生成代码为准
lucide-react	无版本钉死	须用官方 **`0.4xx`**；勿用 npm 上 **`^1.8.0`** 撞名包（缺少 `Github` 等导出）	写入规则 / handoff，避免再踩坑

本 TechDesign **v1.2** 已与当前仓库 M0 实施一致；§10 TD-01~08 仍有效。后续仅改版本号与「变更说明」，不删除 v1.1 历史结论。

v1.2 增补（**2026-04-23**，文档可读性）

- **§1.2**：新增 **1.2.0**（三条链路总览表、时序图符号说明、Mermaid 注意点）；在 **1.2.1～1.2.3** 各节后增加白话说明、参与者表、分步/双时间线表，并各附一张补充流程图（与原有顺序图并存，便于线性阅读）。
- **§1.3**：新增 **1.3.0**、与 §1.2 的对照表、分层职责表、依赖规则表及「纵向切片」示例说明；不修改原容器分层示意图，仅增加文字与表格释义。

v1.2 增补（2026-04-23，GitHub Mermaid 兼容）

- 修正 **§1.1** subgraph id（GH_Platform / OR_Platform）与 class 引用一致；& 改为「与」。
- **§1.2.2** 时序图：/api/jobs/:id ⟶ {id}；i18n/<lang>-<ts> ⟶ 纯文字分支名描述。
- **§1.2.3** 补充流程图：API[/api/webhooks/github] 改为带引号矩形节点，避免梯形语法与路径斜杠冲突。
- **§3.1** ER：Model ⟶ AiModel；repositoryId FK,UK ⟶ 单行 UK；简化 ignoreGlobs 与模型 id 示例字符串。
- **§5.1** 流程图：节点文案去掉 {{P1}} 双花括号。
- **§7.1**：preview-<sha> 改为不含尖括号的描述。
- **§7.2**：CI/CD 流程图中 Lint[eslint + tsc --noEmit] 未引号包裹导致 **--** 被解析为边语法；改为全部 **id["…"]** 节点并略去易踩坑的
。
- 新增 Cursor 规则 **.cursor/rules/docs-mermaid-github.mdc**，后续改 docs/**/*.md 中的 Mermaid 时遵循该文件。